Threads

Lecture 10

Computer Networks and Distributed Programming

Where we are: one process per client

Your server can already keep several clients going at once: one `fork` per connection, one child each.

One client types a line. How does it reach the other two?

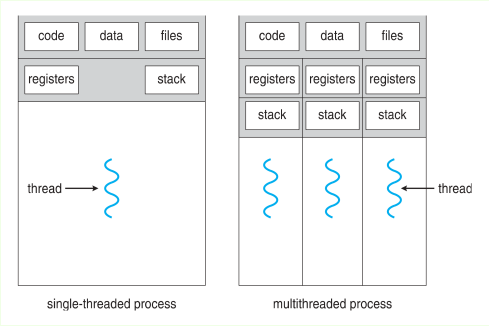
It does not. At the `fork` each child got a private copy of the memory, so there is no such thing as "the list of connected clients".

- You could send the line between the processes — and that is a second program, with its own protocol, that you have not written.
- Every connection also costs an address space, page tables and a new entry in the scheduler's tables.
- What we want instead: another flow of control inside the same process, sharing everything by default.

One process, several flows of control

Definition

A thread is one flow of control inside a process: its own stack and its own registers, everything else in common with its peers.



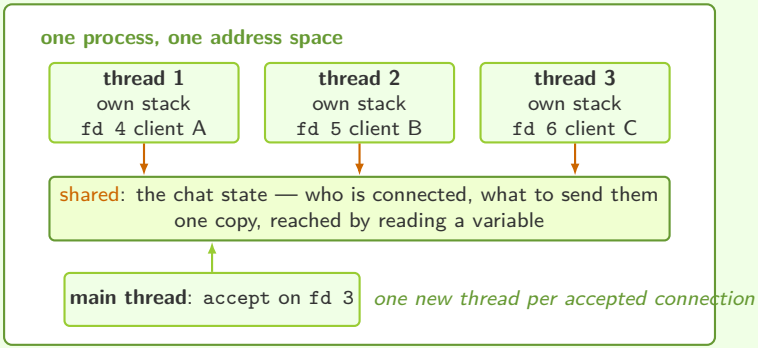
The process is no longer the thing that runs: it is the container. What the scheduler runs is a thread, and until today every process had one.

What is private, what is shared

Private to each thread	**Shared by every thread of the process**
thread id, registers, program counter	the code, and every global variable
its own **stack**, so its locals	the heap: whatever `malloc` returned
the signal **mask**	the **descriptor table**: sockets and files
`errno`	the disposition of each signal — the handlers
scheduling priority	pid, ppid, working directory, credentials

The left column is short on purpose: what a thread owns is the state of one computation, and nothing else. In particular a descriptor opened by one thread is fd 4 for all of them.

The same picture, with sockets



One descriptor table, so one `close` ends a connection; and the chat state is a variable, not a message anybody has to send.

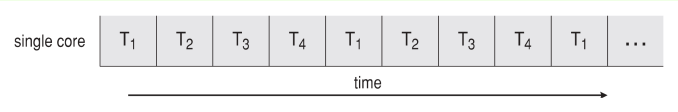
What the sharing buys, and what it costs

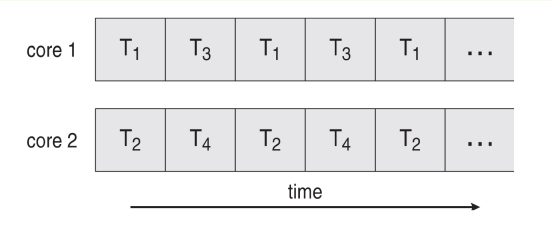
- No IPC: the list of connected clients is a variable: one thread writes it, the others read it, and nothing is transmitted.
- Cheaper: creating a thread copies no address space, and switching between two of them changes no page tables.
- Independent blocking: a thread waiting inside `read` holds up only itself, provided the kernel knows it exists.
- The price: nothing separates them: one thread that corrupts a shared structure corrupts it for everybody.
- And a thread that dereferences a null pointer takes down the whole server, clients and all — a child process took down only itself.

A forking server is isolated by construction. A threaded one is not, and that is what the second half of this lecture is about.

Concurrency is not parallelism

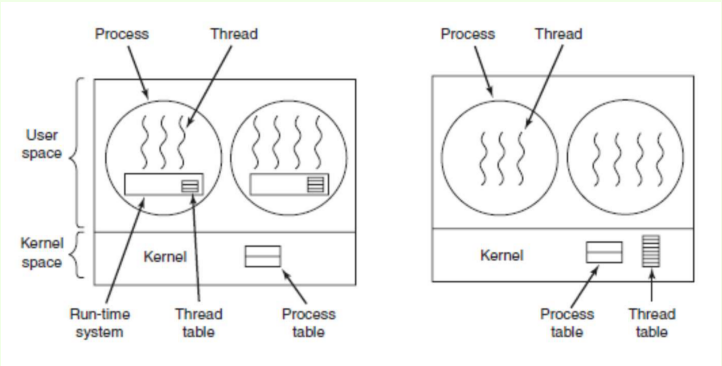
Concurrency: several tasks in progress at once. Parallelism: several tasks executing in the same instant.





A server needs the first one: its threads spend their lives waiting on the network, not computing. The second is a bonus the hardware may provide.

Where the threads are known



- Left: the library does everything in user space: switching is a function call, but the kernel sees one runnable thing.
- Right: the kernel has a table of its own: it schedules each thread, and blocks each one separately.

Three mappings, and the one we get

- Many-to-one: every user thread on a single kernel thread. Switching is cheap, but one blocking call stops all of them, and two cores are never used.
- One-to-one: one kernel thread per user thread: real blocking, real parallelism, and a kernel object for each. This is Linux, and therefore us.
- Many-to-many: m user threads scheduled onto n kernel threads: the general answer, complicated enough that the systems which tried it went back.

So in your server every thread is blocked and scheduled on its own — and every thread is a kernel object, which is why you cannot have a million of them.

pthread_create

```c
#include <pthread.h>

int pthread_create(pthread_t *tid, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
```

- `tid` is an output: the identity of the new thread, of opaque type `pthread_t`. A thread asks for its own with `pthread_self()`.
- `attr` the attributes, or `NULL` for the defaults — which is what you want until you need a detached thread.
- `start_routine` where the new thread begins: it does not continue from here, it calls that function.
- `arg` the single argument that function receives.

Compile with `gcc -pthread`: it links the library and sets the flags the headers expect.

Errors do not travel through errno

Every call so far returned -1 and set `errno`. The pthread functions return the error code themselves and leave `errno` alone.

```c
int err = pthread_create(&tid, NULL, worker, arg);
if (err != 0)
    fprintf(stderr, "pthread_create: %s\n", strerror(err));  /* not perror */
```

- So `perror` after a pthread call prints whatever unrelated failure happened last: the call never touched `errno`.
- The reason is historical — a global would be ambiguous with threads — and `errno` is in fact per thread today.
- `pthread_create` fails for real: `EAGAIN` when the system will not give you another thread, and a server must survive it.

The start function and its one argument

```
struct job { int fd; unsigned long n; };   /* what one thread needs to work */

static void *worker(void *arg) {
    struct job *j = arg;                   /* the void * comes back as it went */
    serve(j->fd, j->n);
    close(j->fd);
    free(j);                               /* this thread owns it, so it frees it */
    return NULL;                           /* the same as pthread_exit(NULL) */
}

struct job *j = malloc(sizeof *j);
j->fd = fd; j->n = ++served;
pthread_create(&tid, NULL, worker, j);   /* one pointer, anything behind it */
```

One argument only, of type `void *`: more than one value means a struct, and that struct must still exist when the thread reads it.

The argument must not be a variable you reuse

```c
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    pthread_create(&tid, NULL, worker, &fd);    /* one variable, every thread */
}
```

Two clients connect a millisecond apart. Which descriptor does each thread serve?

Whatever is in `fd` when it looks. The second `accept` may already have overwritten it: two threads on one connection, and a connection nobody serves.

- Give each thread its own object — one `malloc` per connection, freed by the thread that receives it.
- It works in testing, because in testing the clients arrive one at a time.

Two threads and a main

```c
static void *say(void *arg) {
    printf("%s: tid %lu\n", (char *) arg, (unsigned long) pthread_self());
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, say, "first");
    pthread_create(&t2, NULL, say, "second");
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Nothing says which of the two prints first, and running it twice may answer differently. `pthread_t` is opaque: printing it is a convenience of this system, not a portable one.

When main returns, everybody dies

Remove the two joins. What does the program print?

Often nothing. `return` from `main` is `exit`, and `exit` ends the process — threads that were about to print included.

```
$ ./hello            without the joins:  sometimes one line, sometimes none
second: tid 139903206737600
```

- `exit` called by any thread ends every thread: there is no version of it that ends only the caller.
- Either join what you started, or, if `main` has nothing left to do, end it with `pthread_exit(NULL)`: the process then lives until its last thread ends.
- A `sleep` in `main` is not a way to wait for a thread: it is a bet on how long the work takes.

Three ways a thread ends

- return: from the start function: the value returned is the thread's exit value, and it is the ordinary way.
- pthread_exit(value): the same thing from anywhere, including from a function called ten frames deeper.
- Cancelled: another thread asked for its termination; the exit value is then `PTHREAD_CANCELED`.
- Not a way: `exit`, `_exit`, or a signal whose action is to terminate: those end the whole process.

Whatever the way, the value left behind must outlive the thread — its stack does not.

pthread_join

```c
int pthread_join(pthread_t tid, void **retval);

void *status;
pthread_join(tid, &status);      /* blocks until that thread ends */
pthread_join(tid, NULL);           /* the exit value does not interest us */
```

- It waits for one named thread: there is no `pthread_join(-1)`, no "whichever ends first" as with `wait`.
- If that thread has already ended it returns immediately, exactly like waiting for a child that is already a zombie.
- And that is the analogy to keep: a thread that ended and was not joined still occupies its resources.

The value must outlive the thread

```c
static void *local(void *arg) {
    struct pair p = { 1, 2 };            /* on this thread's stack */
    return &p;                           /* which is gone in a moment */
}
static void *heap(void *arg) {
    struct pair *p = malloc(sizeof *p);
    p->a = 5; p->b = 6;
    return p;                            /* the joiner reads it, then frees it */
}
```

```
thread 1: a=1 b=2
master  : a=1075156600 b=1077281896      the stack was reused
thread 3: a=5 b=6
master  : a=5 b=6
```

A global or the heap; never the stack. The compiler will not stop you, and the first run often prints the right numbers.

Detached threads

A thread nobody will ever join should say so, and then it releases everything the moment it ends.

```
pthread_detach(tid);                    /* from another thread, or ... */
pthread_detach(pthread_self());         /* ... by the thread itself, first thing */

pthread_attr_t attr;                    /* or decided at creation */
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
pthread_create(&tid, &attr, worker, j);
pthread_attr_destroy(&attr);
```

- Threads are joinable by default, so a server that forgets this leaks one thread's worth of bookkeeping per client served.
- Detached is irreversible in practice: from then on `pthread_join` on it is an error, and its exit value is lost.

One thread per connection, in outline

```c
for (;;) {
    int *fd = malloc(sizeof *fd);          /* one per connection, freed by the thread */
    *fd = accept(listen_fd, NULL, NULL);
    if (*fd < 0) { perror("accept"); free(fd); continue; }

    pthread_t tid;
    int err = pthread_create(&tid, NULL, worker, fd);
    if (err != 0) {                        /* out of threads: refuse this one */
        fprintf(stderr, "pthread_create: %s\n", strerror(err));
        close(*fd); free(fd); continue;
    }
    pthread_detach(tid);                   /* nobody is ever going to join it */
}
```

Notice what is missing: no `close` in the accepting thread. There is one descriptor, not two, so whoever closes it ends the connection.

Watching the threads from outside

```
$ gcc -pthread -o chatd chatd.c
$ ./chatd 5000 &
$ ps -o pid,lwp,nlwp,stat,comm -L -p $(pgrep chatd)
  PID   LWP NLWP STAT COMMAND
 5310  5310    4 Sl   chatd        the main thread, in accept
 5310  5311    4 Sl   chatd
 5310  5312    4 Sl   chatd        one per connected client
 5310  5313    4 Sl   chatd
```

- One pid, four LWPs: the kernel schedules each one, and `top -H` shows them consuming CPU separately.
- `kill` still addresses the process, because a signal is sent to a pid — which is the subject of a later frame.

Processes and threads, side by side

Processes	Threads	And the difference
`fork`	`pthread_create`	the child resumes here, the thread starts in a function
`exit`	`pthread_exit`	`exit` in a thread ends the process
`waitpid`	`pthread_join`	no “any of them”: you name the thread
`kill`	`pthread_kill`	the disposition is one per process
`getpid`	`pthread_self`	the id is opaque, only comparable
zombie	a thread nobody joined	same leak, same cure

The names line up, and it is a good map — except that processes are isolated and threads are not.

The counter that loses increments

```c
static int count = 0;                    /* one variable, all the threads */

static void *bump(void *arg) {
    for (int i = 0; i < 100000; i++)
        count++;
    return NULL;
}
/* main: create eight of them, join all eight, print count */
```

```
$ ./bump ; ./bump ; ./bump
count = 431027       expected 800000
count = 800000       and sometimes it is right, which is worse
count = 512663
```

No thread is wrong and no line of C is wrong. And it is not a question of two cores: on one core the same increments are lost, because a thread can be interrupted in the middle of one.

Where the increment goes missing

`count++` is one character. How many machine operations is it?

Three: load, add one, store. A thread can be taken off the processor between any two of them.

Thread A	**Thread B**	`count`
load $\rightarrow$ 7		7
	load $\rightarrow$ 7	7
add, store 8		8
	add, store 8	8

Two increments, one of them gone. Whenever two threads may touch the same variable and one of them writes, the result depends on timing — and we need a way to forbid that overlap.

Not every function may be called by two threads

Your globals are not the only shared state: some library functions keep their own, in a static buffer you never see.

- `strtok` remembers where it stopped; two threads parsing two lines walk over each other's position.
- `gethostbyname`, `ctime`, `inet_ntoa` return a pointer to one buffer, overwritten by the next caller.
- Some have a reentrant twin that takes the buffer from you: `strtok_r`, `rand_r`. Prefer them, and prefer `getaddrinfo`, which was designed this way.
- `errno` is not one of these: it is per thread, so nobody else's failure ever appears in yours.

The manual page says *MT-Safe* or it does not, and it is worth the look before a function goes inside a worker.

Threads and signals

- The disposition is one per process: one `sigaction`, and it holds for every thread. There is no per-thread handler.
- The mask is one per thread: each decides what it blocks, with `pthread_sigmask`.
- A signal sent to the process is delivered to any one thread that does not block it — and you do not get to know which.
- `pthread_kill(tid, sig)` names a thread; but if the disposition is to terminate, the whole process still dies.

```
1 signal(SIGPIPE, SIG_IGN);    /* the write on a closed connection returns EPIPE */
```

So a threaded server usually blocks signals everywhere and lets one thread wait for them.

A thread calls fork

One of eight threads calls `fork`. How many threads has the child?

One: the one that called. The other seven do not exist there — but everything they had done to the memory does.

- Whatever they held stays as it was: a half-updated structure stays half updated, and a lock stays locked forever, since its owner is not there to release it.
- So the child of a threaded process may only do the safe things — in practice, `exec` something, immediately.
- `exec` is unaffected: it replaces the program, and the new one starts with the single thread that called it.

Cancelling a thread, and why a server rarely does

```
pthread_cancel(tid);              /* asks for termination; does not wait */
pthread_join(tid, &status);       /* status == PTHREAD_CANCELED */
```

- By default cancellation is deferred: it takes effect at a cancellation point — `read`, `write`, `accept`, `sleep` — so a thread busy computing does not notice for a while.
- The thread then stops where it stopped: its buffer, its descriptor and anything it had locked stay as they were.
- Which is why a server asks instead of ordering: a `volatile sig_atomic_t` flag the workers test between requests, or simply close the descriptor and let their `read` return.

What a thread costs, and how many you can have

- Creating one is roughly an order of magnitude cheaper than a `fork`: no address space to duplicate, no page tables to copy.
- Switching between two threads of one process does not change the address space, so the caches and the translation buffer survive it.
- But each thread reserves a stack — on Linux 8 MB of address space by default, and you can ask for less at creation.
- And each is a kernel object with its own kernel stack: thousands are ordinary, hundreds of thousands are not.

So one thread per client is fine for a chat with a classroom on it, and not fine at ten thousand connections — where a fixed set of workers takes over, and one thread watches many connections at once.

Summary

- A thread owns its stack, its registers and its signal mask; everything else — globals, heap, descriptors — belongs to the process.
- So threads share state by reading a variable, which is exactly what a chat server could not do with processes.
- `pthread_create` takes a function and one argument, and that argument must belong to the thread it is given to.
- The pthread calls return an error code, not -1; and a thread that ended is joined or detached, or it leaks.
- `exit` and `main` returning end every thread; only `pthread_exit` and `return` end just one.
- And the price of sharing is `count++`: correct code, wrong result, whenever two threads write the same thing.

Try it

Two short programs

One: take `n` from the command line, create `n` threads that each sleep a random number of seconds, then increment a global counter and print it; wait for all of them. Two: the same, with no sleep and each thread incrementing 100 000 times.

```
$ ./threads 8          run each one twenty times and count the answers
```

Then answer: are there race conditions in the first program? And why does the second one show them and the first one hides them?